

Coder avec Claude Code

Développement assisté par IA — méthodes et pratiques professionnelles

IA	LLM	Prompts	Claude Code
----	-----	---------	-------------

Vérification	Workflow	Git
--------------	----------	-----

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le fonctionnement des grands modèles de langage pour le code
- ✓ Maîtriser l'art du prompt engineering appliqué au développement logiciel
- ✓ Développer un workflow de vérification systématique du code généré par IA
- ✓ Identifier les forces et les limites de l'assistance IA au développement
- ✓ Intégrer Claude Code dans un workflow Git professionnel
- ✓ Aborder les enjeux éthiques et pratiques du développement assisté par IA

Table des matières

Chapitre 1 — Les grands modèles de langage pour le code

- 1.1 Comment fonctionne un LLM
- 1.2 La fenêtre de contexte
- 1.3 Ce que LLM signifie vraiment
- 1.4 Hallucinations et confabulations

Chapitre 2 — Prompt engineering pour le code

- 2.1 La méthode CROC
- 2.2 Contexte — donner les bonnes informations
- 2.3 Contraintes — délimiter le scope
- 2.4 Itération et raffinement

Chapitre 3 — Vérification et validation

- 3.1 Les trois types d'erreurs IA
- 3.2 Checklist de revue systématique
- 3.3 Tests comme filet de sécurité
- 3.4 La règle du '0 erreur TypeScript'

Chapitre 4 — Workflow Git avec l'IA

- 4.1 Commits atomiques après chaque prompt
- 4.2 Messages de commit descriptifs
- 4.3 Branches pour les fonctionnalités risquées
- 4.4 Review avant chaque push

Chapitre 5 — Forces, limites et éthique

- 5.1 Ce que l'IA fait bien
- 5.2 Ce que l'IA fait mal
- 5.3 Responsabilité du développeur

5.4 L'avenir du développement assisté

Les grands modèles de langage pour le code

1.1 Comment fonctionne un LLM

Un LLM (Large Language Model) est un réseau de neurones entraîné sur d'immenses quantités de texte — incluant des milliards de lignes de code. Son fonctionnement fondamental est la prédiction du token suivant : étant donné la séquence de tokens précédents (votre prompt + la conversation), le modèle calcule une distribution de probabilité sur tous les tokens possibles et en sélectionne un. Ce processus se répète pour chaque token généré.

Cette description technique a une implication profonde : le modèle ne 'comprend' pas le code au sens humain. Il n'exécute pas le code mentalement pour vérifier qu'il fonctionne. Il produit le code qui lui semble le plus probable étant donné le contexte — le code qui ressemble à du code correct dans des millions d'exemples similaires vus pendant l'entraînement. C'est pourquoi le code généré peut être syntaxiquement parfait mais logiquement incorrect.

Définition — Token

Un token est l'unité de base des LLMs — généralement un mot, une partie de mot, ou un signe de ponctuation. 'développement' est typiquement 2-3 tokens. 'const' est 1 token. Un caractère unicode rare peut être plusieurs tokens. La longueur d'une conversation en tokens détermine si elle rentre dans la fenêtre de contexte du modèle.

1.2 La fenêtre de contexte

La fenêtre de contexte est la quantité maximale de texte (en tokens) que le modèle peut 'voir' simultanément. Tout ce qui est en dehors de cette fenêtre est invisible pour le modèle. Les modèles modernes ont des fenêtres de 100 000 à 200 000 tokens, ce qui correspond à environ 75 000 - 150 000 mots — suffisant pour contenir un projet complet de taille moyenne.

En pratique, les performances du modèle dégradent souvent sur les informations au milieu d'un très long contexte. Pour un usage optimal, fournissez le contexte le plus pertinent et le plus concis possible : les fichiers directement concernés par la tâche, les interfaces TypeScript pertinentes, les conventions du projet. Évitez d'inonder le contexte avec des fichiers non pertinents.

1.3 Hallucinations et confabulations

Le terme 'hallucination' désigne les cas où un LLM génère des informations fausses mais présentées avec assurance. Pour le code, cela peut prendre plusieurs formes : appeler une méthode qui n'existe pas dans la bibliothèque, utiliser une option de configuration qui n'existe pas, ou décrire le comportement d'une API de façon incorrecte.

Les hallucinations sont plus fréquentes sur les APIs récentes (le modèle a moins d'exemples d'entraînement), les bibliothèques obscures, et les détails très spécifiques. La contre-mesure est la vérification systématique : ne jamais appeler une méthode sans avoir vérifié qu'elle existe dans la documentation ou le code source de la bibliothèque.

Prompt engineering pour le code

2.1 La méthode CROC

Un prompt efficace pour le code contient quatre éléments : Contexte (les fichiers et fonctionnalités existants), Résultat attendu (ce que le code doit faire précisément), Objectif (le pourquoi — la fonctionnalité utilisateur), et Contraintes (les règles à respecter). Plus ces éléments sont précis, plus le code généré sera correct du premier coup.

■ *Prompt vague – réponse générique et potentiellement incorrecte*

Ajoute une fonctionnalité de recherche de produits.

■ *Prompt CROC – réponse ciblée et exploitable*

CONTEXTE :

- Fichier : backend/src/routes/products.ts (route Express existante)
- La table 'products' a les colonnes : id, tenant_id, name, price, stock_quantity, deleted_at
- Le middleware requireAuth attache req.tenant.id à chaque requête

OBJECTIF :

Permettre aux marchands de rechercher leurs produits par nom depuis l'écran de gestion du catalogue.

RÉSULTAT ATTENDU :

Modifier la route GET /products pour accepter un paramètre query 'q'.

Si q est présent : filtrer les produits dont le nom contient q (ILIKE).

Si q est absent : comportement actuel inchangé.

CONTRAINTES :

- TypeScript strict, 0 erreur tsc --noEmit
- Toujours filtrer par tenant_id (req.tenant.id)
- Exclure les produits soft-deleted (deleted_at IS NULL)
- Limiter à 50 résultats maximum pour la recherche
- Pas de nouveau fichier – modifier uniquement products.ts

2.2 Contexte — donner les bonnes informations

Le contexte est la partie la plus importante d'un prompt. Un modèle qui ne voit pas les fichiers existants peut générer du code parfaitement valide en isolation, mais incompatible avec le projet. Les éléments de contexte les plus utiles sont : les interfaces TypeScript des données manipulées, le schéma SQL des tables concernées, le contenu des fichiers à modifier, et les conventions établies dans le projet.

```
# Éléments de contexte à toujours fournir :  
  
1. FICHIERS CONCERNÉS  
'Voici le contenu actuel de backend/src/routes/orders.ts : [contenu]'  
  
2. INTERFACES TYPESCRIPT  
'L'interface Order est définie ainsi : [interface]'  
  
3. SCHÉMA SQL (si migration nécessaire)  
'La table orders a ces colonnes : [liste]'  
  
4. DERNIÈRE MIGRATION (contexte des migrations précédentes)  
'La dernière migration est 013_adjust_product_stock.sql'  
  
5. ERREUR EXACTE (si débogage)  
'tsc retourne cette erreur : [erreur exacte avec fichier et ligne]'
```

2.3 Itération et raffinement

Un prompt parfait du premier coup est l'exception, pas la règle. La pratique recommandée est d'itérer : commencer par une version simple, vérifier le résultat, et affiner dans les messages suivants. Chaque itération apporte plus de précision. Si la réponse s'éloigne de ce qui est attendu, fournir un exemple du comportement souhaité est souvent plus efficace que d'ajouter des explications textuelles.

Vérification et validation

3.1 Les trois types d'erreurs IA

Le code généré par IA peut contenir trois types d'erreurs, de gravité croissante. Le premier type, les erreurs syntaxiques et de compilation, sont détectables immédiatement par `tsc --noEmit` ou le compilateur. Ce sont les plus faciles à corriger. Le deuxième type, les erreurs logiques silencieuses, passent la compilation mais produisent un comportement incorrect : une requête sans filtre `tenant_id`, un calcul de total incorrect, une condition inversée. Elles nécessitent une lecture attentive du code et des tests. Le troisième type, les erreurs d'intégration, surviennent quand le code est correct isolément mais incompatible avec le reste du projet : une migration qui supprime une colonne encore utilisée, une fonction qui change la signature d'une interface partagée.

3.2 Checklist de revue systématique

La revue du code généré par IA doit être aussi rigoureuse que la revue de code en équipe. Voici la checklist à appliquer après chaque prompt qui génère ou modifie du code.

CHECKLIST DE REVUE – après chaque prompt Claude Code

COMPILATION

- `npx tsc --noEmit` → 0 erreur TypeScript
- Aucun import manquant ou inexistant

LECTURE DU CODE

- Chaque fichier modifié lu ligne par ligne
- La logique correspond au comportement décrit
- Aucune condition inversée (`>` vs `<`, `===` vs `!==`)

SÉCURITÉ (pour chaque requête DB)

- Filtrage par `tenant_id` présent
- Inputs utilisateur validés avant utilisation
- Pas de données sensibles dans les logs
- Tokens stockés chiffrés

TESTS

- Les tests existants passent toujours (`npm test`)
- Comportement testé manuellement dans l'app

GIT

- Commit atomique avec message Conventional Commits
- git diff relu avant commit

3.3 La règle du '0 erreur TypeScript'

Une règle absolue dans le développement avec IA : ne jamais commiter du code qui contient des erreurs TypeScript. Les erreurs tsc signalent soit un vrai problème logique, soit une incompatibilité avec le projet. Les laisser s'accumuler crée une dette technique qui rend les prochains prompts moins fiables — le modèle voit du code 'erroné' et peut en reproduire les patterns.

Workflow Git avec l'IA

4.1 Le cycle prompt → vérification → commit

La discipline Git est encore plus importante quand on travaille avec l'IA. La vitesse de génération de code peut pousser à accumuler plusieurs prompts sans commiter — c'est une mauvaise pratique. Si un prompt introduit un bug, il est beaucoup plus difficile de l'identifier quand 5 prompts ont été appliqués ensemble. La règle est : 1 prompt validé = 1 commit.

```
# Le cycle de travail recommandé

# 1. Écrire le prompt (méthode CROC)
# 2. Appliquer les fichiers générés
# 3. VÉRIFIER :
npx tsc --noEmit
git diff --name-only # Quels fichiers ont changé ?
# Lire chaque fichier modifié

# 4. Tester le comportement
npm test
# + test manuel si UI

# 5. Commiter
rm -f .git/index.lock # Précaution
git add -A
git commit -m "feat(products): add search by name with ILIKE filter"

# Ne JAMAIS accumuler plusieurs prompts sans commiter
# Si quelque chose va mal, git reset --hard HEAD repart d'un état propre
```

4.2 Messages de commit descriptifs

La convention Conventional Commits est particulièrement utile quand on travaille avec l'IA : elle force à formuler ce que le code fait (pas comment), ce qui sert de vérification mentale. Si vous ne pouvez pas rédiger un message clair, c'est souvent le signe que le prompt couvrait trop de choses à la fois.

Forces, limites et éthique

5.1 Ce que l'IA fait bien

Les LLMs sont particulièrement efficaces pour les tâches répétitives et à forte structure. Les routes Express suivent toutes le même pattern — une fois que le modèle a vu une route, il peut en générer d'autres avec très peu d'erreurs. Les migrations SQL, les composants React, les tests d'après des fixtures existantes — toutes ces tâches sont bien dans la zone de confort d'un LLM. L'IA est aussi excellente pour expliquer du code complexe, suggérer des refactorings, et débbugger quand on lui fournit le message d'erreur exact.

5.2 Ce que l'IA fait mal

L'IA est moins fiable pour la logique métier complexe qui requiert une compréhension profonde du domaine. Un algorithme de calcul de commission variable selon plusieurs paramètres, une logique de disponibilité des créneaux, une formule financière précise — ces cas nécessitent une vérification humaine rigoureuse. L'IA peut aussi introduire des vulnérabilités de sécurité subtiles : oublier de vérifier le `tenant_id`, omettre la validation d'un input, créer une race condition dans une transaction.

5.3 Responsabilité du développeur

L'utilisation de l'IA pour générer du code ne transfère pas la responsabilité. Le développeur qui commite du code — qu'il l'ait écrit lui-même ou qu'il provienne d'une IA — en est entièrement responsable. 'L'IA l'a généré' n'est pas une justification acceptable en cas de faille de sécurité, de violation de données, ou de bug en production. C'est la raison pour laquelle la vérification systématique n'est pas optionnelle.

■ Bonne pratique

Le meilleur développeur de l'ère IA n'est pas celui qui fait le moins de travail — c'est celui qui sait exactement quoi déléguer à l'IA, comment guider la génération, et surtout comment vérifier rigoureusement le résultat. La compétence de vérification est au moins aussi importante que la compétence de prompting.

5.4 L'avenir du développement assisté

L'assistance IA au développement n'en est qu'à ses débuts. Les modèles deviennent plus précis, les fenêtres de contexte plus larges, les outils d'intégration plus sophistiqués. Des agents capables d'exécuter du code, de lire les erreurs de compilation, et d'itérer automatiquement commencent à émerger. Dans ce contexte, les compétences de haut niveau — architecture, sécurité, compréhension des besoins utilisateurs, qualité du code — deviennent plus importantes, pas moins. L'IA automatise

l'écriture de code boilerplate ; la pensée critique reste humaine.

■ À retenir

- ✓ Un LLM prédit le token suivant — il ne 'comprend' pas le code, il le génère par probabilité
- ✓ La méthode CROC : Contexte + Résultat + Objectif + Contraintes pour chaque prompt
- ✓ Trois types d'erreurs IA : compilation (facile), logique silencieuse (dangereuse), intégration
- ✓ Règle absolue : 0 erreur tsc --noEmit avant tout commit
- ✓ Cycle : 1 prompt → vérification → commit atomique — ne jamais accumuler
- ✓ La responsabilité du code généré par IA reste entièrement celle du développeur

— ■ Exercices

Exercice 1 — Analyse de prompts

Évaluer la qualité de prompts existants et les améliorer.

- Prendre 3 prompts passés utilisés avec Claude Code
- Identifier les éléments CROC présents et manquants dans chacun
- Réécrire chaque prompt selon la méthode CROC complète
- Comparer les résultats obtenus avec les deux versions

Exercice 2 — Bug hunt IA

Identifier les erreurs dans du code généré par IA.

- Demander à Claude Code d'implémenter une route sans contexte de tenant
- Identifier les erreurs de type 1 (tsc), type 2 (logique), type 3 (intégration)
- Corriger chaque erreur en expliquant pourquoi c'est un problème
- Rédiger le prompt amélioré qui aurait évité ces erreurs

Exercice 3 — Sprint simulé

Implémenter une petite fonctionnalité de bout en bout avec le bon workflow.

- Écrire le prompt CROC pour la fonctionnalité
- Appliquer le code, vérifier (tsc + lecture + tests)
- Commiter avec Conventional Commits
- Bilan : nombre de prompts, erreurs trouvées, corrections apportées

Bibliographie & Ressources

LLMs et IA

- anthropic.com/research — recherches Anthropic sur les LLMs
- docs.anthropic.com/en/docs/build-with-claude/prompt-engineering — guide officiel
- simonwillison.net — blog de Simon Willison sur les LLMs pour développeurs

Prompt Engineering

- learnprompting.org — guide prompt engineering (open source)
- promptingguide.ai — DAIR.AI Prompt Engineering Guide

Workflow et productivité

- conventionalcommits.org — spécification Conventional Commits
- gitmoji.dev — emoji pour les messages de commit